

Successive maximum refinement by an ordered list of functionals

`MathlibAnnex.lexicographicRefine`

def

Defines a nonempty compact set of survivors after maximizing finitely many functionals in order.

Statement

For a nonempty compact set K in a real normed space and a finite ordered list $\ell = (\ell_1, \dots, \ell_r)$ of continuous linear functionals, successively retain the points maximizing ℓ_1 , then those maximizing ℓ_2 on the retained set, and so on. The resulting set is denoted $\mathcal{R}_\ell(K)$.

Definition

Let $K_\ell = \{x \in K : \ell(x) = \max_{y \in K} \ell(y)\}$. The complete recursion is

$$\mathcal{R}_\emptyset(K) = K, \quad \mathcal{R}_{(\ell, \eta)}(K) = \mathcal{R}_\eta(K_\ell).$$

The maximum of each later functional is taken on the current retained set. Changing the order can therefore change the resulting set.

Assumptions

The ambient space is any real normed vector space. The set K is nonempty and compact, and the list is finite and ordered. No finite-dimensionality or point-separation assumption is needed for this definition.

Conclusion

The result is a nonempty compact set. For the empty list it is K itself. A singleton conclusion requires a separate point-separation hypothesis; this definition does not select a single point.

Each maximum slice is nonempty compact by continuity on a nonempty compact set. Thus every recursive step is defined. Later lemmas show that the final set lies in K and that all survivors have the same value under every functional in the list. If the list separates points, those equalities force the final set to be a singleton.

Main citations

- Exact declaration and its source — `MathlibAnnex.lexicographicRefine`
- A nonempty compact maximum slice — `MathlibAnnex.NonemptyCompacts.refine`
- Refinement remains inside its original set — `MathlibAnnex.lexicographicRefine_subset`
- All survivors have the same functional values —
`MathlibAnnex.apply_eq_of_mem_lexicographicRefine`

- Point separation forces at most one survivor — `MathlibAnnex.lexicographicRefine_subsingleton`

Lean source signature (exact)

```
noncomputable def lexicographicRefine :
  List (E →L[ $\mathbb{R}$ ]  $\mathbb{R}$ ) → TopologicalSpace.NonemptyCompacts E →
    TopologicalSpace.NonemptyCompacts E
| [], K => K
|  $\ell$  :: ls, K => lexicographicRefine ls (refine K  $\ell$ )
```

In the source	Mathematical meaning
<code>{E : Type u} [NormedAddCommGroup E] [NormedSpace $\mathbb{R}$ E]</code>	An arbitrary real normed space; no finite-dimensionality assumption.
<code>List (E →^L[$\mathbb{R}$] $\mathbb{R}$)</code>	The finite ordered list ℓ of continuous real-linear functionals.
<code>TopologicalSpace.NonemptyCompacts E → TopologicalSpace.NonemptyCompacts E</code>	Input nonempty compact set K and output nonempty compact set $\mathcal{R}_\ell(K)$; the result is a set, not a point.

In the source	Mathematical meaning
<code> [], K => K</code>	The full empty-list clause $\mathcal{R}_\emptyset(K) = K$.
<code> ℓ :: ls, K => lexicographicRefine ls (refine K ℓ)</code>	The full recursive clause: first retain $K_\ell = \{x \in K : \ell(x) = \max_K \ell\}$, then process the remaining list. The cited <code>refine</code> bundles that same maximum slice with nonemptiness and compactness.

▼ Exact surrounding binder context (separate excerpt)

```
noncomputable section

open Set
open TopologicalSpace

namespace MathlibAnnex

universe u

variable {E : Type u} [NormedAddCommGroup E] [NormedSpace  $\mathbb{R}$  E]
```

▼ Exact content identity

Declaration: `MathlibAnnex.lexicographicRefine`

Accepted content SHA-256: 6958ff7e499de016a5c4ec9b2d88c97a1a6a2fa0b9f7c199ed0e6adcb03777f6

Accepted source guide SHA-256: 0e928aff04d7d85e813d48de09ff7f81540307222a38f660ed03fb6d5ef8b898

Source commit: 437e6e46228bbb8e91211ebded349d7a30020e73